Facultatea de Științe Aplicate

Anul III, grupa 1333

Cercetări Operaționale

Proiect Programare Liniară

DUMITRACHE D. Iustin-Florin 1333a
GEANTĂ H. Grigorie 1333a
IONESCU L.M. Cristina-Maria 1333a
PODOLEANU G. Minodora-Elena 1333a

Cuprins

1 Introducere 4

2 Modelul matematic al unei probleme de programare liniară(PL) 5

2.1 Forma generală(FG) (Bibic, 2021) 5

2.2 Forma matriceală(FM) (Bibic, 2021) 5

2.3 Forma canonică(FC) 6

3 Transformarea problemei de programare liniară în forma standard (PLS) 7

4 Pseudocod 9

4.1 Link cod 11

5 Model de optimizare a profitului: Analiza resurselor limitate în recondiționarea tech 11

5.1 Introducere 11

5.2 Modelul matematic (PL) 12

5.3 Standardizarea modelului (PLS) 13

5.4 Rezolvarea prin Algoritmul Simplex Primal 13

5.5 Validarea soluției optime 14

5.6 Metoda grafică 14

5.7 Interpretarea rezultatelor și validarea multidirecțională 15

6 Concluzii 16

7 Bibliografie 17

1 Introducere

Într-o lume în care resursele sunt limitate, iar cerințele sunt din ce în ce mai complexe, capacitatea de a lua decizii optime devine un avantaj competitiv esențial. Fie că este vorba despre maximizarea profitului unei companii sau minimizarea costurilor într-un sistem logistic, matematica oferă instrumente precise pentru a transforma aceste obiective în soluții concrete.

Conceptul de programare liniară a apărut în prima jumătate a secolului al XX-lea, în contextul necesităților militare din timpul celui de-Al Doilea Război Mondial. În acea perioadă, era necesară optimizarea utilizării resurselor logistice, precum transportul, distribuția materialelor și planificarea producției. În anul 1947, matematicianul George Dantzig a pus bazele metodei simplex, utilizată pentru rezolvarea problemelor de planificare cu structură liniară. Ulterior, s-a demonstrat că un număr foarte mare de probleme din diverse domenii pot fi modelate și rezolvate prin această metodă.

Problemele de optimizare, respectiv cele de maxim și de minim, apar frecvent în cele mai diverse domenii ale matematicii pure și aplicate, constituind un instrument esențial în analiza și rezolvarea situațiilor reale. În domeniul economic, aceste probleme sunt deosebit de importante, deoarece firmele urmăresc în mod constant maximizarea profiturilor sau minimizarea costurilor, iar deciziile trebuie luate în condiții de resurse limitate . (Nica, Cercetări Operaționale: Capitolul 11)

Aceste preocupări au condus la dezvoltarea cercetărilor operaționale, un domeniu interdisciplinar al matematicii aplicate care utilizează modele matematice, metode analitice și tehnici de optimizare pentru fundamentarea deciziilor eficiente. Cercetările operaționale permit transformarea problemelor reale complexe în modele matematice riguroase, facilitând identificarea soluțiilor optime în diverse contexte economice și tehnice.

În acest cadru, programarea liniară reprezintă o metodă de optimizare bazată pe modelarea matematică a unor probleme decizionale, în care se urmărește determinarea valorilor unor variabile de decizie astfel încât o funcție obiectiv să fie maximizată sau minimizată, în condițiile respectării unui set de restricții exprimate prin relații liniare.

Pentru a evidenția caracterul aplicativ al programării liniare, pot fi menționate câteva exemple reprezentative. O problemă cunoscută este problema firmei, care constă în stabilirea cantităților optime de produse ce trebuie realizate astfel încât profitul să fie maxim, în condițiile limitării resurselor disponibile, precum materiile prime sau timpul de lucru .

O altă aplicație clasică este problema dietei, în care se urmărește alegerea unei combinații de alimente care să asigure necesarul de nutrienți la un cost minim.

De asemenea, problema transportului are ca obiectiv determinarea modului optim de distribuire a unor cantități de produse de la mai multe surse către mai multe destinații, astfel încât cererea să fie satisfăcută cu cost minim.

Dintre metodele utilizate pentru rezolvarea acestor probleme, algoritmul simplex se remarcă prin eficiența și aplicabilitatea sa largă, fiind unul dintre cele mai importante instrumente ale optimizării liniare.

Prin urmare, programarea liniară reprezintă un instrument fundamental al cercetărilor operaționale, având un impact major asupra modului în care sunt luate deciziile în economie, industrie și tehnologie. Capacitatea sa de a transforma probleme reale în modele matematice și de a furniza soluții optime o face indispensabilă în contextul actual.

2 Modelul matematic al unei probleme de programare liniară(PL)

Modelul matematic al unei probleme de programare liniară reprezintă transpunerea unei situații reale într-un sistem de relații matematice, cu scopul de a determina soluția optimă în condiții de resurse limitate. Acest model este alcătuit din trei componente esențiale:
a)variabilele problemei- reprezintă elementele necunoscute ale problemei, a căror valoare trebuie determinată astfel încât să se obțină rezultatul optim.
b)funcția obiectiv –care poate fi de min sau de max
c) sistemul de constrângeri/restricții-reprezintă totalitatea restricțiilor în care se desfășoară activitatea tehnico-economică. (Bibic, 2021)

2.1 *Forma generală(FG)* (Bibic, 2021)

1. opt $f(x)=\sum_{j=1}^{n} c_j x_j = c_1x_1 + c_2x_2 + \cdots + c_nx_n$

2. $\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n \\ \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n \end{cases} \begin{pmatrix} \leq \\ = \\ \geq \end{pmatrix} \begin{matrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{matrix} \quad sau \quad \sum_{j=1}^{n} a_{ij}x_j \begin{pmatrix} \leq \\ = \\ \geq \end{pmatrix} b_i, i = \overline{1,m}$

3. $x_1, x_2, \dots, x_n \begin{cases} \geq 0 \\ \leq 0 \\ \in \mathbb{R} \end{cases}$

2.2 *Forma matriceală(FM)* (Bibic, 2021)

1) opt f(x)=cx
2) $Ax \begin{pmatrix} \leq \\ = \\ \geq \end{pmatrix} b$
3) $x \begin{cases} \geq 0 \\ \leq 0 \\ \in \mathbb{R} \end{cases}$

,unde:

- Optimul:

  opt=max/min $\Longrightarrow$ max(f(x))=-min(-f(x))
  sup/inf min(f(x))=-max(-f(x))

- Coeficienții funcției obiectiv

  $c=(c_1\ c_2 \dots c_n)$

Din punct de vedere aplicativ,avem:

c: -beneficii(pentru problemele de max) ;

-costuri(pentru problemele de min).

- Variabilele problemei =>vectorul care reprezintă planul/programul de producție:

$$x = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}$$

- Resursele disponibile de producție:

$$b = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{pmatrix}$$

- Matricea coeficienților tehnologici:

$$A=\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix} \in \mathbb{M}_{m\times n}(\mathbb{R})$$

Mai departe ne vom folosi de forma matriceală(FM) pentru a defini și celelalte forme.

2.3 Forma canonică(FC)

Este forma în care restricţiile sunt concordante şi toate variabilele sunt supuse condiţiei de nenegativitate. (Grigoraș)

OBSERVAȚIE!

În FG a PL,dacă:

Optim	Restricția ≥	Restricția ≤
MAX	RD	RC
MIN	RC	RD

RC=relație de concordanță

RD=relație de discordanță

FC când opt=MIN	**FC când opt=MAX**	
1)min f(x)=cx	1)max f(x)=cx	
2)Ax≥b	2)Ax≤b	(Bibic, 2021)
3)x≥0	3)x≥0	

3 Transformarea problemei de programare liniară în forma standard (PLS)

Pentru a putea aplica eficient algoritmul simplex, orice problemă de programare liniară (PL) trebuie adusă într-o formă standardizată, denumită forma standard (PLS). Această transformare nu este doar o formalitate matematică, ci un pas esențial care permite algoritmului să funcționeze corect și sistematic.

Forma generală a unei PLS

Este forma în care toate restricţiile sunt exprimate prin egalităţi şi toate variabilele sunt supuse condiţiei de nonnegativitate. (Grigoraș)

1) opt f(x) = cx

2) Ax=b (Bibic, 2021)

3) x ≥ 0

Importanţa acestei forme particulare rezultă din faptul că metoda de rezolvare a problemelor de programare liniară care va fi expusă mai departe cere ca problema să fie în această reprezentare. În consecinţă, o problemă (PL) care are şi restricţii inegalităţi va fi înlocuită - în vederea rezolvării ei - cu o alta în care toate restricţiile sunt egalităţi. Noua problemă, numită forma standard a problemei (PL) şi notată PLS, se construieşte prin următoarele reguli de standardizare: (Nica, Cercetări Operaționale: Capitolul 11)

Regula1 (Bibic, 2021)

Se aplică condițiilor 3) din PL și are scopul de a aduce toate variabilele la forma x≥0,adică variabile nenegative

→ Dacă x ≤0,spunem că x este variabilă nepozitivă,ceea ce nu convine,deci se va face următoarea transformare:

$$\begin{cases} notăm\ x = x', undex' \geq 0 \\ în\ relațiile\ 1)\ și\ 2)\ din\ PL\ se\ înlocuiește\ \boxed{x = x'} \end{cases}$$

→ Dacă x∈ ℝ,spunem că x este variabilă liberă, ceea ce nu convine, deci se va face următoarea transformare:

$$\begin{cases} notăm\ x = x' - x'', unde\ x', x'' \geq 0 \\ în\ relațiile\ 1)\ și\ 2)\ din\ PLS\ se\ înlocuiește\ x = x' - x'' \end{cases}$$

Regula 2 (Bibic, 2021)

Se aplică condiției 2) din PL și are scopul de a transforma semnul tuturor restricțiilor în “=”.

→ Pentru inegalități de tipul ≤,se adaugă o variabilă de compensare y(dar doar în partea stângă a inecuației din sitemul de restricții) și vom obține:

- "≤" $devine$ " = "
- $\begin{cases} \text{în relația 1) din PL} \Longrightarrow opt\ f(x) = c \cdot x + 0 \cdot y \\ \text{în relația 3) din PL} \Longrightarrow y \geq 0 \end{cases}$

→ Pentru inegalități de tipul ≥, se scade o variabilă de compensare y și se adaugă o variabilă de penalizare z (dar doar în partea stângă a inecuației din sitemul de restricții) și vom obține:

- "≥" $devine$=
- $\begin{cases} \text{în relația 1) din PL} \Longrightarrow opt\ f(x) = c \cdot x + 0 \cdot y \pm M \cdot z \\ \text{în relația 3) din PL} \Longrightarrow y, z \geq 0 \end{cases}$

→ Pentru "=",se adaugă o variabilă de penalizare z (dar doar în partea stângă a ecua din sitemul de restricții) și vom obține:

- "=" devine "="
- $\begin{cases} \text{în relația 1) din PL} \Longrightarrow opt\ f(x) = c \cdot x \pm M \cdot z \\ \text{în relația 3) din PL} \Longrightarrow z \geq 0 \end{cases}$

În cazul restricțiilor de tip egalitate, deși acestea sunt deja în formă standard, nu permit identificarea directă a unei baze inițiale pentru algoritmul simplex. Din acest motiv, se introduce o variabilă de penalizare, care asigură existența unei soluții de pornire. Ulterior, această variabilă este forțată să devină nulă prin metoda Big M.

OBSERVAȚIE!

Metoda Big M

Metoda Big M reprezintă o tehnică de extindere a algoritmului simplex, utilizată pentru rezolvarea problemelor de programare liniară care conțin restricții de tip „mai mare sau egal" sau egalități. Aceasta introduce în funcția obiectiv termeni de penalizare de forma ±M, unde M este un număr foarte mare, ce tinde spre infinit, iar z reprezintă variabila de penalizare. Scopul acestor termeni este de a forța eliminarea variabilelor de penalizare din soluția optimă.

M este un număr foarte mare;

Semnul depinde de optimul problemei:

$$\begin{cases} opt = MAX \Longrightarrow -Mz \\ opt = MIN \Rightarrow +Mz \end{cases}$$

4 Pseudocod

```
FUNCTIE test_optimalitate(delta, optim):
    epsilon <- 1e-8
    DACA (optim == "max") ATUNCI:
        RETURNEAZA (toti d <= epsilon pentru d in delta)
    ALTFEL:
        RETURNEAZA (toti d >= -epsilon pentru d in delta)
FUNCTIE criteriu_intrare_baza(delta, c, optim):
    epsilon <- 1e-8
    DACA (optim == "max") ATUNCI:
        val_tinta <- max(delta)
    ALTFEL:
        val_tinta <- min(delta)
    indici <- [i pentru i, v in delta daca |v - val_tinta| < epsilon]
    castigator <- indici[0]
    PENTRU fiecare idx in indici[1:]:
        DACA (optim == "max" SI c[idx] > c[castigator]) ATUNCI:
            castigator <- idx
        DACA (optim == "min" SI c[idx] < c[castigator]) ATUNCI:
            castigator <- idx
    RETURNEAZA castigator
FUNCTIE criteriu_iesire_baza(A, termeni_liberi, index_intrare, indici_baza):
    m <- numarul de linii din A
    col_pivot <- A[:, index_intrare]
    rapoarte <- vector de dimensiune m, initializat cu +infinit
    epsilon <- 1e-8
    PENTRU fiecare linie i in [0, m):
        DACA (col_pivot[i] > epsilon) ATUNCI:
            rapoarte[i] <- termeni_liberi[i] / col_pivot[i]
    theta_min <- min(rapoarte)
    DACA (theta_min == +infinit) ATUNCI:
        RETURNEAZA None
    egali <- [i pentru i in [0, m) daca |rapoarte[i] - theta_min| < epsilon]
    DACA (lungime(egali) == 1) ATUNCI:
        RETURNEAZA egali[0]
    RETURNEAZA i din egali care maximizeaza indici_baza[i]
FUNCTIE pivotare(A, m, n, i_piv, j_piv):
    A_np  <- copie float a lui A
    piv   <- A_np[i_piv, j_piv]
    A_nou <- matrice de zerouri (m x n)
    A_nou[i_piv, :] <- A_np[i_piv, :] / piv
    PENTRU fiecare linie i in [0, m):
        DACA (i != i_piv) ATUNCI:
            A_nou[i, :] <- A_np[i, :] - A_np[i, j_piv] * A_nou[i_piv, :]
    RETURNEAZA A_nou
FUNCTIE verificare_optim_infinit(A, delta, optim):
    epsilon <- 1e-8
    PENTRU fiecare coloana k si delta[k] din delta:
        DACA (optim == "max" SI delta[k] > epsilon) SAU (optim == "min" SI delta[k] < -epsilon) ATUNCI:
            DACA (toti A[i, k] <= epsilon) ATUNCI:
                RETURNEAZA ADEVARAT, k
    RETURNEAZA FALS, None
FUNCTIE verificare_optim_multiplu(A, delta, indici_baza):
    n <- numarul de coloane din A
    PENTRU fiecare coloana j in [0, n):
        DACA (j NU se afla in indici_baza) ATUNCI:
            DACA (str(round(delta[j], 10)) == "0.0") ATUNCI:
                DACA (exista i astfel incat A[i, j] > 1e-8) ATUNCI:
                    RETURNEAZA ADEVARAT, j
    RETURNEAZA FALS, None
```

```
FUNCTIE simplex_primal(A, b, c, optim, x_names, n_orig):
    A, b, c          = copii float ale matricelor de intrare
    iteratie          <- 0
    deja_gasit_optim <- FALS
    MAX_ITERATII      <- 1000
    CAT TIMP (iteratie < MAX_ITERATII) executa:
        iteratie <- iteratie + 1
        indici_baza <- gaseste_indici_identitate(A)
        cb <- c[indici_baza]
        zj[j] <- cb · A[:, j] pentru fiecare coloana j
        delta[j] <- c[j] - zj[j] pentru fiecare coloana j
        Z <- cb · b
        AFISEAZA(A, indici_baza, b, delta, Z)
        infinit, idx_k <- verificare_optim_infinit(A, delta, optim)
        DACA (infinit == ADEVARAT) ATUNCI:
            RETURNEAZA None
        este_optim <- test_optimalitate(delta, optim)
        DACA (este_optim == ADEVARAT) ATUNCI:
            multiplu, idx_alt <- verificare_optim_multiplu(A, delta, indici_baza)
                      DACA (multiplu == ADEVARAT) SI (NON deja_gasit_optim) ATUNCI:
                j_in <- idx_alt
                deja_gasit_optim <- ADEVARAT
            ALTFEL:
                RETURNEAZA A, b, indici_baza, cb
        ALTFEL:
            j_in <- criteriu_intrare_baza(delta, c, optim)
        i_out <- criteriu_iesire_baza(A, b, j_in, indici_baza)
        DACA (i_out este NULL) ATUNCI:
            RETURNEAZA None
        tabel <- [A | b]
        tabel_nou <- pivotare(tabel, i_out, j_in)
        A, b <- tabel_nou[:, :-1], tabel_nou[:, -1]

    RETURNEAZA None

```

```
//MAIN
# PAS 1 - Conditia (1): Functia obiectiv
AFISEAZA "PROGRAMARE LINIARA - SIMPLEX PRIMAL"
AFISEAZA "[PAS 1 - Conditia (1)] FUNCTIA OBIECTIV"

s1 <- CITESTE "Introduceti functia: "
optim, n_vechi, x, c <- parsare_conditie_1(s1)

AFISEAZA "Tip optimizare:", optim
AFISEAZA "Variabile:", x
AFISEAZA "Coeficienti c:", c

# PAS 1 - Conditia (2): Restrictiile in forma standard
AFISEAZA "[PAS 1 - Conditia (2)] RESTRICTIILE"

(A, b, semne, x, c_nou, A_vechi, x_vechi, c_vechi, semne_vechi) <- parsare_conditie_2(x, optim, c)

c_arr <- conversie_float(c_nou)
b_initial <- copie(b)

AFISEAZA "Variabile extinse:", x
AFISEAZA "Coeficienti c extinsi:", c_nou
AFISEAZA "Matricea A (forma standard):", A
AFISEAZA "Vectorul b:", b

# PAS 1 - Conditia (3): Conditii de nenegativitate
AFISEAZA "[PAS 1 - Conditia (3)] CONDITII DE NENEGATIVITATE"
```

```
parsare_conditie_3(x)

# PAS 2 + PAS 3 - Algoritmul Simplex Primal
AFISEAZA "PAS 2 + PAS 3 - ALGORITMUL SIMPLEX PRIMAL"

rezultat <- simplex_primal(A, b, c_arr, optim, x, n_vechi)

# AFISARE SOLUTIE FINALA + PAS 3 (Validare): Proba matematica
DACA (rezultat nu este None) ATUNCI:
    A_fin, b_fin, indici_baza, cb_fin <- rezultat

    AFISEAZA "SOLUTIA FINALA"
    z_optim <- produs_scalar(cb_fin, b_fin)
    AFISEAZA "Valoarea optima Z =", z_optim

    PENTRU i de la 0 la n_vechi - 1:
        DACA (i se afla in indici_baza) ATUNCI:
            pos <- pozitia lui i in indici_baza
            valoare <- b_fin[pos]
            status <- "Baza"
        ALTFEL:
            valoare <- 0.0
            status <- "Secundara"
        AFISEAZA x_vechi[i], valoare, status

    # PAS 3 - VALIDARE MATEMATICA (PROBA)
    AFISEAZA "PAS 3 - VALIDARE MATEMATICA (PROBA)"

    S <- coloanele din A_initial_ext corespunzatoare indici_baza
    verificare_b <- produs_matriceal(S, b_fin)

    DACA (verificare_b este aproximativ egal cu b_initial) ATUNCI:
        AFISEAZA "PROBA REUSITA: S * X_B == b initial"
    ALTFEL:
        AFISEAZA "EROARE: Diferente detectate la proba"

ALTFEL:
    AFISEAZA "Problema nu are solutie finita"

```

4.1 Link cod

https://github.com/IustinFlorinDumitrache/Programare-liniara-proiect-1

5 Model de optimizare a profitului: Analiza resurselor limitate în recondiționarea tech

5.1 Introducere

În contextul actual al sustenabilității, managementul unui start-up specializat în recondiționarea echipamentelor tehnologice își propune maximizarea profitului săptămânal. Compania și-a structurat portofoliul în jurul a două categorii principale de produse, realizate integral din componente reutilizate:

- Eco-Phone **(x_1):** Smartphone-uri recondiționate prin integrarea de baterii și ecrane reciclate.

- Tab-Green **(x_2):** Tablete destinate sectorului educațional, asamblate din piese reutilizate.

Provocarea cu care se confruntă este specifică oricărei afaceri moderne: gestionarea unui timp limitat și a unui stoc redus de componente. Astfel, se urmărește determinarea volumului optim de producție săptămânal pentru fiecare tip de unitate, astfel încât să se maximizeze profitul prin utilizarea eficientă a resurselor limitate ale laboratorului.

Pentru a identifica acest mix optim de producție, au fost analizate limitările operaționale ale procesului de recondiționare. Capacitatea laboratorului este condiționată de disponibilitatea utilajelor specializate și de stocul finit de piese recuperate, după cum urmează:

- Capacitatea de diagnosticare: această etapă necesită 2 ore pentru un Eco-Phone și 5 ore pentru un Tab-Green, dintr-un total de 30 de ore disponibile săptămânal.
- Procesul de micro-lipire și reparație: un Eco-Phone consumă 1 oră, în timp ce un Tab-Green necesită 3 ore de manoperă specializată, resursa totală fiind de 20 de ore săptămânal.
- Gestiunea componentelor critice: Fiecare unitate produsă (indiferent de tip) utilizează 1 cip de control din stocul limitat de 15 unități pe săptămână.

Din punct de vedere financiar, analiza rentabilității indică faptul că un Eco-Phone generează un profit unitar de 100 u.m., în timp ce un Tab-Green contribuie cu 150 u.m. la profitul total al start-up-ului.

Pe baza acestor premise, modelul de optimizare matematică va fi construit pentru a echilibra cerințele tehnice cu obiectivele financiare ale companiei.

5.2 Modelul matematic (PL)

Pentru a identifica soluția optimă, transpunem variabilele de business în limbaj matematic. Acest model va servi drept bază pentru algoritmul de optimizare, transformând restricțiile fizice în inecuații liniare.

Definirea variabilelor de decizie:

- x_1 =numărul de unități Eco-Phone produse săptămânal.
- x_2 =numărul de unități Tab-Green produse săptămânal.

1. *Funcția obiectiv*

Deoarece urmărim maximizarea beneficiului total, funcția obiectiv este de tip MAX:

1. $\max f(x) = 100x_1 + 150x_2$
2. *Restricțiile*

$$\begin{cases} 2x_1 + 5x_2 \leq 30 \\ x_1 + 3x_2 \leq 20 \\ x_1 + x_2 \leq 15 \end{cases}$$

3. *Condiția de nenegativitate*

$$x_1, x_2 \geq 0$$

5.3 Standardizarea modelului (PLS)

Pentru a aplica Algoritmul Simplex Primal (ASP), este necesară aducerea problemei la forma standard (PLS) prin transformarea inegalităților în egalități. Introducem variabilele de compensare $y_1, y_2, y_3 \geq 0$.

1. *Funcția obiectiv*

$$\max f = 100x_1 + 150x_2 + 0y_1 + 0y_2 + 0y_3$$

2. *Restricțiile*

$$\begin{cases} 2x_1 + 5x_2 + y_1 = 30 \\ x_1 + 3x_2 + y_2 = 20 \\ x_1 + x_2 + y_3 = 15 \end{cases}$$

3. *Condiția de nenegativitate*

$$x_1, x_2, y_1, y_2, y_3 \geq 0$$

5.4 Rezolvarea prin Algoritmul Simplex Primal

Vom inițializa algoritmul prin construirea tabelului simplex asociat formei canonice, urmând să efectuăm iterațiile necesare pentru îmbunătățirea valorii funcției obiectiv până la atingerea punctului optim.

OPT=MAX			x_1	x_2	y_1	y_2	y_3	VAR
			$c_1 = 100$	$c_2 = 150$	$c_3 = 0$	$c_4 = 0$	$c_5 = 0$	c_j
C_B	B	x_B	$a_1 \downarrow$	$a_2 \downarrow$	a_3	a_4	a_5	
0	$a_3 \uparrow$	30	2	5	1	0	0	I_0
0	a_4	20	1	3	0	1	0	
0	a_5	15	1	1	0	0	1	
		$z^0 = 0$	$\Delta_1 = 100$	$\Delta_2 = 150$	$\Delta_3 = 0$	$\Delta_4 = 0$	$\Delta_5 = 0$	$TO(I_0)$ ✗
150	a_2	6	2/5	1	1/5	0	0	I_1
0	a_4	2	-1/5	0	-3/5	1	0	
0	$a_5 \uparrow$	9	3/5	0	-1/5	0	1	
		$z^1 = 900$	$\Delta_1 = 40$	$\Delta_2 = 0$	$\Delta_3 = -30$	$\Delta_4 = 0$	$\Delta_5 = 0$	$TO(I_1)$ ✗
150	a_2	0	0	1	1/3	0	-2/3	

0	a_4	5	0	0	-2/3	1	1/3	I_2
100	a_1	15	1	0	-1/3	0	5/3	
		$z^2 = 1500$	$\Delta_1 = 0$	$\Delta_2 = 0$	$\Delta_3 = -50/3$	$\Delta_4 = 0$	$\Delta_5 = -200/3$	$TO(I_2)$✓

5.5 Validarea soluției optime

Conform testului de optimalitate (TO) aplicat în iterația I_2 se observă că toți $\Delta_j \leq 0$. Acest fapt indică atingerea punctului de optim, algoritmul încetând procesul iterativ (STOP).

Soluția optimă (PL) : $x_{max} = \begin{pmatrix} x_1 = 15 \\ x_2 = 0 \end{pmatrix}$, $f_{max} = 1500$.

Pentru confirmarea validității acestui rezultat, se vor efectua următoarele verificări:

1. $\forall x_j \geq 0$,condiție îndeplinită, $x_1 = 15, x_2 = 0$.
2. $f_{max} = 100 \times 15 + 150 \times 0 = 1500$.
3. $\begin{pmatrix} 30 \\ 20 \\ 15 \end{pmatrix} = \begin{pmatrix} 5 & 0 & 2 \\ 3 & 1 & 1 \\ 1 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 5 \\ 15 \end{pmatrix} \Leftrightarrow \begin{pmatrix} 30 \\ 20 \\ 15 \end{pmatrix} = \begin{pmatrix} 30 \\ 20 \\ 15 \end{pmatrix}$.

Toate cele trei condiții de verificare sunt îndeplinite simultan. Astfel, modelul nu este doar o proiecție matematică, ci o strategie operațională validă, gata de a fi implementată în cadrul managementului start-up-ului.

5.6 Metoda grafică

Interpretarea geometrică a modelului confirmă rezultatele obținute prin algoritmul Simplex Primal, oferind o perspectivă spațială asupra restricțiilor tehnologice. În sistemul de coordonate zona admisibilă (S) este reprezentată de poligonul convex format sub intersecția celor trei drepte de frontieră:

→ Dreapta albastră: $2x_1 + 5x_2 = 30$.
→ Dreapta verde: $x_1 + 3x_2 = 20$.
→ Dreapta roșie : $x_1 + x_2 = 15$.

Analiza graficului indică faptul că punctul optim se află în vârful A(15,0). Se observă că în acest punct, dreapta roșie și cea albastră se intersectează pe axa orizontală, semnalând saturarea completă a acestor resurse. Dreapta verde se situează deasupra acestui punct, confirmând vizual existența variabilei de compensare y_2.

5.7 Interpretarea rezultatelor și validarea multidirecțională

Corelarea rezultatelor obținute prin algoritmul Simplex Primal cu analiza geometrică oferă o imagine clară asupra modului în care resursele laboratorului dictează performanța financiară a start-up-ului. Din punct de vedere matematic, profitul maxim de 1500 u.m. este localizat în vârful A(15,0) al poligonului convex de soluții admisibile, punct ce marchează intersecția dintre capacitatea maximă de diagnosticare și stocul disponibil de componente critice. Această configurație indică o saturare completă a două dintre cele trei restricții impuse, demonstrând că orice deviere de la acest mix de producție ar conduce la o utilizare ineficientă a resurselor și, implicit, la un profit inferior.

Validarea soluției nu s-a rezumat doar la calculul manual, ci a fost susținută de o abordare multidirecțională. Interpretarea grafică a confirmat vizual că, în timp ce diagnosticarea și componentele sunt utilizate la capacitate maximă, resursa dedicată micro-lipirii prezintă un excedent, reprezentat prin variabila de compensare y_2 . Această observație este esențială pentru management, semnalând faptul că forța de muncă specializată nu reprezintă, în acest moment, un blocaj în calea creșterii profitului.

Mai mult, precizia acestor deducții a fost reconfirmată prin utilizarea algoritmilor de optimizare numerică într-un script Python. Identitatea rezultatelor obținute digital cu cele calculate prin metoda Simplex atestă nu doar corectitudinea tehnică a modelului, ci și versatilitatea acestuia ca instrument de simulare. Această convergență a metodelor, de la rezolvarea algebrică și geometrică până la cea computațională, transformă modelul dintr-o simplă analiză teoretică într-o strategie operațională robustă, capabilă să se adapteze instantaneu la schimbările de pe piața recondiționărilor tehnologice.

```
==================================================
SOLUTIA FINALA
==================================================

Valoarea optima Z (MAX) = 1500.0

Variabila  Valoare     Status in baza
--------------------------------------
x1         15.0        Baza (variabila principala)
x2         0.0         Baza (variabila principala)
```

```
PAS 3 — VALIDARE MATEMATICA (PROBA)
==============================================================

VIZUALIZARE PROBA (S * X_B = b_initial):
     Matricea S          X_B          Rezultat       b_init
--------------------------------------------------------------
[  5.0000    0.0000    2.0000]       [  0.0000]       [ 30.0000]       [ 30.0000]
[  3.0000    1.0000    1.0000]   x   [  5.0000]   =   [ 20.0000]  ==?  [ 20.0000]
[  1.0000    0.0000    1.0000]       [ 15.0000]       [ 15.0000]       [ 15.0000]

[V] PROBA REUSITA: S * X_B == b initial.

Process finished with exit code 0
```

6 Concluzii

Analiza efectuată în cadrul acestei lucrări demonstrează că programarea liniară reprezintă un pilon esențial al cercetărilor operaționale, oferind un cadru riguros pentru transformarea problemelor decizionale complexe în modele matematice cu soluții precise. Prin parcurgerea etapelor de definire a formelor generală, matriceală și canonică, s-a evidențiat necesitatea standardizării (PLS) pentru aplicarea algoritmului Simplex, proces care asigură tratarea sistematică a tuturor restricțiilor de tip inegalitate prin introducerea variabilelor de compensare.

Studiul de caz aplicat asupra start-up-ului de recondiționare tehnologică a scos în evidență valoarea practică a acestor concepte, demonstrând cum limitările fizice, precum timpul de diagnosticare, manopera specializată și stocul finit de componente, pot fi echilibrate pentru atingerea unui obiectiv financiar. Rezultatele obținute prin Algoritmul Simplex Primal au indicat faptul că profitul maxim săptămânal de 1500 u.m. este atins prin producția exclusivă a 15 unități de Eco-Phone. Această soluție subliniază un aspect critic al optimizării: în condiții de resurse strict limitate, mixul de producție cel mai eficient este cel care saturează resursele critice, în acest caz fiind vorba despre capacitatea de diagnosticare și stocul de cipuri de control.

Validarea multidirecțională a rezultatului, realizată atât prin interpretarea geometrică a poligonului convex de soluții admisibile, cât și prin verificarea computațională în Python, atestă acuratețea calculelor și robustețea modelului propus.

În final, lucrarea demonstrează că utilizarea unor instrumente matematice precum metoda Simplex nu reprezintă doar un exercițiu teoretic, ci o strategie operațională validă care permite managementului să ia decizii fundamentate pe date, optimizând fluxurile de producție într-un context economic marcat de resurse limitate.

7 Bibliografie

Bibic, S.-M. (2021). Cercetări Operaționale: Programare Liniară (Cursuri și Seminare).

6 Probleme de Programare Liniară. (n.d.). Retrieved from https://ro.scribd.com/doc/310040773/6-Probleme-de-Programare-Liniara

Anthropic. (2026). Claude 3. Retrieved from https://claude.ai

Google. (2026). Gemini (Versiunea 1.5 Flash). Retrieved from https://gemini.google.com

Grigoraș, G. (n.d.). *Programarea Liniară.* Retrieved from http://gheorghe-grigoras.ieeia.tuiasi.ro/SDOE/5_Programarea%20LINIARA.pdf

Marin, M. (n.d.). *Logica și Programare Logică (C-05).* Retrieved from https://staff.fmi.uvt.ro/~mircea.marin/lectures/CSI/C-05.pdf

Masgras, V. (n.d.). *Cercetări Operaționale.* Retrieved from https://ro.scribd.com/document/566822640/V-Masgras-Cercetari-Operationale

Mitruț, D. (n.d.). *Baze ale Cercetării Operaționale.* Retrieved from https://ctipub-my.sharepoint.com/personal/minodora_podoleanu_stud_fsa_upb_ro/Documents/Meetings

Nica, V. (n.d.). *Cercetări Operaționale: Capitolul 11.* Retrieved from https://asecib.ase.ro/Nica/CO/BCO/capitolul11.pdf

Nica, V. (n.d.). *Cercetări Operaționale: Capitolul 17.* Retrieved from https://asecib.ase.ro/Nica/CO/BCO/capitolul17.pdf

OpenAI. (2026). ChatGPT (Versiunea GPT-4). Retrieved from https://chat.openai.com